Quality Plan

1. Project Description

Project overview

Team 1 develops a small Python web application for managing simple tasks. The system consists of:

- a **FastAPI** backend exposing REST endpoints,
- a **SQLite** database for persistence,
- a **Streamlit** frontend for basic interaction,
- a **GitHub Actions** pipeline for automated verification.

The application is intentionally small and educational. Its purpose is not to deliver a production-grade product, but to help the team practice **quality engineering**, **continuous integration**, **automated checks**, and **evidence-based quality control**.

Scope

In scope:

- create, read, update, delete tasks;
- mark task status;
- simple filtering/searching;
- API documentation through FastAPI/OpenAPI;
- local SQLite persistence;
- minimal Streamlit interface for demo usage;
- automated tests and CI checks.

Out of scope:

- authentication/authorization;
- distributed deployment;
- production database;
- advanced frontend design;
- real scalability guarantees;
- full security hardening for internet exposure.

Key features

- REST endpoints for task management;
- persistent storage in SQLite;
- validation of request payloads;
- basic error handling for missing/invalid records;
- Streamlit view for entering and listing tasks;
- generated API docs;
- CI pipeline with quality gates.

Constraints

- implementation time is about **2 weeks**;
- small student team, limited engineering capacity;
- toy project with learning focus;
- lightweight stack only;

- quality controls must be automated as much as possible.

Quality strategy

Because the project is small, the quality strategy emphasizes:

1. **simple, enforceable thresholds** instead of heavy process;
2. **automation first** through pre-commit and GitHub Actions;
3. **fast feedback** on every push and pull request;
4. **visible evidence** through tool outputs and CI logs;
5. **minimum viable documentation** sufficient for review and demo.

The team is expected to keep the system easy to understand, easy to test, and stable enough for demo use. For this reason, maintainability and reliability are the main priorities, followed by basic security and lightweight performance verification.

---

2. Quality Requirements Table

Attribute	Metric	Threshold	Tool	CI Gate?	Rationale
Maintainability	Cyclomatic complexity	**< 8 per function**	radon	Yes	Higher than course floor would be too loose for such a small codebase
Maintainability	Maintainability Index	**≥ 70 average**	radon	No	Tracked as warning metric, not hard blocker
Maintainability	Style conformance	**0 errors**	flake8	Yes	Small projects should be fully style-clean
Maintainability	Duplication	**No obvious copy-paste blocks > 15 lines**	manual review / optional ruff-similarities	No	Manual because project is small
Maintainability	Type hygiene	**No new mypy errors in checked modules**	mypy	No	Encouraged, not blocking due to short timeframe
Reliability	Unit/integration tests	**100% pass**	pytest	Yes	Non-negotiable
Reliability	Line coverage	**≥ 75%**	pytest-cov	Yes	Higher than course minimum because scope is small
Reliability	API contract behavior	Core CRUD endpoints return expected status codes and schemas	pytest	Yes	Covered by automated API tests
Reliability	Defect leakage	**0 known blocker/critical bugs at demo**	issue tracker / manual check	Yes (Release)	Demo readiness
Performance	API response time	**P95 < 150 ms** for local CRUD workload	locust	No	Stronger than floor, but advisory due to local variability
Performance	Startup time	App starts locally in **< 10 s**	manual observation	No	Important for demo usability
Security	High-severity vulnerabilities	**0**	bandit	Yes	Required
Security	Medium-severity vulnerabilities	**0 new unresolved medium issues**	bandit	No	Tracked, discussed if any appear
Security	Dependency vulnerabilities	**0 known high vulnerabilities**	pip-audit	Yes	Useful even for toy apps
Documentation	API endpoints documented	**100% FastAPI/OpenAPI coverage for implemented endpoints**	OpenAPI docs	No	Visible in docs UI

Attribute	Metric	Threshold	Tool	CI Gate?	Rationale
Documentation	README completeness	Setup, run, test, lint, architecture, limitations all present	manual review	Yes (PR/Release)	Essential for handover
Usability	Demo path completion	Create/edit/complete/list task flow works without crash	manual test	Yes (Release)	Fits small Streamlit scope

Notes on negotiated thresholds

Compared with the minimum floor, Team 1 is expected to hit:

- **coverage 75% instead of 60%**,
- **complexity < 8 instead of < 10**,
- additional **dependency vulnerability check**,
- explicit **README completeness**,
- explicit **demo-path usability check**.

These are realistic because the feature set is narrow.

3. Quality Gates

Gate	When	What Runs / Is Checked	What Blocks?
Pre-commit	Before push	flake8, bandit, basic tests for changed modules if feasible	flake8 errors, bandit high issues
PR (pre-merge)	On pull request	flake8, pytest, pytest-cov, bandit, pip-audit, code review	test failures, coverage < 75%, lint errors, high vulns, no reviewer approval
Main branch protection	Merge to main	passing CI + 1 review	any required status red
Release gate	Before demo	all CI green + manual smoke test + critical flow works	any red CI status, broken CRUD, broken Streamlit demo, missing README/API docs

Gate policy

- No direct pushes to protected main branch.
- Every PR must have at least one peer review.
- If coverage drops below threshold, the PR is rejected.
- Temporary bypasses are not allowed unless explicitly approved by the quality manager and documented.

4. Tools with Commands

Requirement	Tool	Command
Cyclomatic Complexity	radon	`poetry run radon cc -a -s src/`
Maintainability Index	radon	`poetry run radon mi -s src/`
Style	flake8	`poetry run flake8 src tests`
Tests	pytest	`poetry run pytest`
Coverage	pytest-cov	`poetry run pytest --cov=src --cov-report=term-missing --cov-fail-under=75`
Security (code)	bandit	`poetry run bandit -r src/`
Dependency security	pip-audit	`poetry run pip-audit`
Performance	locust	`poetry run locust -f tests/performance/locustfile.py --headless -u 10 -r 2 -t 1m`
App run (API)	uvicorn	`poetry run uvicorn src.main:app --reload`
App run (UI)	streamlit	`poetry run streamlit run src/ui/app.py`

5. Definition of Done

A feature is done only if:

- code is merged through PR;
- all tests pass;
- coverage remains at or above 75%;
- no flake8 errors;
- no high bandit issues;
- no high dependency vulnerabilities;
- endpoint is visible in OpenAPI if applicable;
- README or doc is updated if behavior changed;
- basic manual demo flow still works.

---

6. CI/CD Implementation Expectations

GitHub Actions pipeline stages

1. checkout and Python setup;
2. Poetry install;
3. lint;
4. tests with coverage;
5. bandit;
6. pip-audit;
7. publish artifacts/logs if useful.

Required checks in GitHub

- `lint`
- `tests`
- `coverage`
- `bandit`
- `dependency-audit`

---

7. Evidence to Collect

The team must preserve:

- CI screenshots or links;
- final coverage report;
- radon summary;
- bandit report;
- short release smoke-test checklist;
- README and OpenAPI evidence.